



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251188
Nama Lengkap	ANGELA REVALINE SETIAWAN
Minggu ke / Materi	15 / Fungsi Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Link Github : https://github.com/angelarevalines/71251188_valine.git

1. Pengertian Rekursif

- **Fungsi Rekursif** adalah fungsi yang berisi dirinya sendiri atau fungsi yang mendefinisikan dirinya sendiri.
- Fungsi Rekursif sering disebut sebagai fungsi yang memanggil dirinya sendiri.
- Fungsi Rekursif juga merupakan fungsi matematis yang berulang dan memiliki pola yang terstruktur.
- Biasanya fungsi ini perlu diperhatikan agar fungsi ini dapat berhenti dan tidak menghabiskan memori.
- Fungsi Rekursif bersifat **unlimited loop** sehingga bisa menyebabkan program hang up.
- 2 blok penting dalam Rekursif :
 - a) **Base Case** = bagian dimana penentu bahwa fungsi rekursif itu akan berhenti.
 - b) **Rekursif Case** = bagian dimana terdapat statement yang akan terus diulang terus-menerus hingga mencapai Base Case.

2. Kelebihan dan Kekurangan

- **KELEBIHAN**
 - 1) Kode program lebih singkat dan elegan.
 - 2) Masalah kompleks dapat di breakdown menjadi sub masalah yang lebih kecil di dalam rekursif.
 - 3) Sesuai untuk struktur data rekursif seperti pohon dan graf.
- **KEKURANGAN**
 - 1) Memakan memori yang lebih besar karena setiap kali bagian dirinya dipanggil maka dibutuhkan sejumlah ruang memori tambahan.
 - 2) Mengorbankan efisiensi dan kecepatan.
 - 3) Fungsi Rekursif sulit dilakukan debugging.
 - 4) Kadang sulit dimengerti (debugging).

3. Bentuk Umum dan Studi Kasus

- Bentuk umum fungsi rekursif

```
laprak.py > ...
1  def fungsi_rekursif(parameter):
2      if base_case(parameter):    #kondisi berhenti
3          return nilai_dasar
4      else:
5          #modifikasi parameter
6          return operasi(fungsi_rekursif(parameter_termodifikasi),parameter)
```

- Contoh 1 (faktorial) :

Algoritmanya :

- 1) Tanyakan n
- 2) Siapkan variabel total untuk menampung hasil perkalian faktorial dan set nilai awal dengan 0
- 3) Loop dari i = 1 hingga n untuk mengerjakan :
- 4) total = total * i
- 5) Tampilkan total

```
laprak.py > ...
1  def faktorial(n):
2      if n == 0 or n == 1:
3          return 1
4      else:
5          return faktorial(n-1) * n
6
7  print(faktorial(5))
```

➤ Contoh 2

```
laprak.py > ...
1  def coba(n):
2      if n == 0: #base case
3          print("UKDW SERU")
4      else:
5          print(n)
6          coba(n-1) #rekursif case
7
8  coba(3)
```

Output :

```
3
2
1
UKDW SERU
```

4. Materi Tambahan

➤ Direct dan Indirect Recursion

Direct = fungsi yang memanggil dirinya sendiri secara langsung.

Contoh :

```
laprak.py > ...
1  #CONTOH DIRECT RECURSION
2
3  def fungsiA():
4      fungsiA
```

Indirect = fungsi A memanggil B, lalu fungsi B memanggil A.

Contoh :

```
laprak.py > ...
1  #CONTOH INDIRECT RECURSION
2
3  def fungsiA():
4      |      fungsiB
5
6  def fungsiB():
7      |      fungsiA
```

➤ **Tail dan Non-Tail Recursion**

Tail Recursion = pemanggilan fungsi rekursif merupakan operasi terakhir.

Contoh :

```
laprak.py > ...
1  #TAIL RECURSION
2
3  def tail_sum(n, acc = 0):
4      |      if n == 0:
5      |          |      return acc
6      |      return tail_sum(n - 1, acc + n)
```

Non-Tail Recursion = ada operasi lagi setelah pemanggilan fungsi rekursif.

Contoh :

```

laprak.py > ...
1  #NON-TAIL RECURSION
2
3  def non_tail_sum(n):
4      if n == 0:
5          return 0
6      return n + non_tail_sum(n - 1)

```

➤ Optimasi Rekursif

Memorization merupakan teknik menyimpan hasil komputasi untuk menghindari perhitungan berulang.

Contoh program kasus **Fibonacci dengan memorization** :

```

laprak.py > ...
1  memo = {}
2  def fibo_memo(bil):
3      if bil in memo:
4          return memo[bil]
5
6      if bil <= 2:
7          return 1
8
9      memo[bil] = fibo_memo(bil - 1) + fibo_memo(bil - 2)
10     return memo[bil]

```

➤ Batas Rekursi di Python

Default recursion limit = 1000

Namun, bisa diubah menggunakan :

```

laprak.py
1  #CARA UBAH BATAS REKURSI
2
3  import sys
4  sys.setrecursionlimit(7000)
5

```

➤ Beberapa Contoh Kesalahan Rekursif

1) Tidak ada Base Case

Maka akan menghasilkan **Recursion Error**

```
laprak.py > ...  
1 #CONTOH KESALAHAN  
2  
3 def salah(n):  
4     return n + salah(n + 2)  
5  
6 #tidak akan berhenti
```

2) Parameter tidak berubah atau berkurang

```
laprak.py > ...  
1 #CONTOH KESALAHAN  
2  
3 def inf(n):  
4     if n > 0:  
5         print(n)  
6         inf(n)  
7  
8 #nilai n tidak berubah
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/angelarevalines/71251188_valine.git

SOAL 1

Source Code :

```
def prima(n, bagi = 2):  
    if n < 2:  
        return False  
    if bagi > n ** 0.5:  
        return True  
    if n % bagi == 0:  
        return False  
    return prima(n, bagi + 1)  
  
#TEST CASE  
print(prima(5))  
print(prima(20))  
print(prima(11))
```

Penjelasan :

1. Buat fungsi bernama **prima** dengan parameter **n** dan **pembagi** (yang telah di set default = 2).
2. Input bilangan adalah **n**, sedangkan pembaginya diuji mulai dari 2 sampai seterusnya.
3. **if n < 2: return False** karena bilangan yang kurang dari 2 jelas **bukan bilangan prima**.

4. Lalu **if bagi > n ** 0.5 : return True** karena jika pembaginya melebihi akar kuadrat **n**, berarti **n** tidak punya pembagi lain yang berarti bilangan tersebut merupakan bilangan prima.
5. **if n % bagi == 0 : return False** karena jika **n** habis dibagi oleh **bagi**, maka **n** bukan bilangan prima.
6. Rekursif terjadi di **return prima(n, bagi + 1)** yang berarti jika belum memenuhi kondisi di atasnya, maka akan dilanjutkan pengecekan dengan **bagi + 1**.

Output :

```
True
False
True
```

SOAL 2

Source Code :

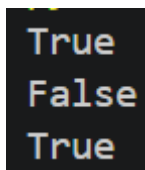
```
def palindrom(kata):  
    kata = kata.lower().replace(" ", "")  
    if len(kata) <= 1:  
        return True  
    return kata[0] == kata[-1] and palindrom(kata[1:-1])  
  
#TESTCASE  
print(palindrom("makam"))  
print(palindrom("halloww"))  
print(palindrom("yey"))
```

Penjelasan :

1. Buat fungsi bernama **palindrom** dengan parameter bernama **kata**.
2. Menggunakan fungsi **lower()** untuk mengubah semua karakter menjadi huruf kecil agar tidak **case sensitive**.
3. Menggunakan fungsi **.replace(" ", "")** untuk menghapus spasi agar kalimat dapat di cek.
4. **If len(kata) <= 1: return True** karena dianggap palindrom.

5. Rekursif terjadi pada **return kata[0] == kata[-1] and palindrom(kata[kata1:-1])** untuk membandingkan karakter pertama dengan karakter terakhir. Jika sama, maka dilanjutkan pengecekan ke substring tanpa karakter pertama dan terakhir.
6. Jika tidak sama, maka langsung mengembalikan False karena bukan palindrom.

Output :



```
True
False
True
```

SOAL 3

Source Code :

```
def hitung_ganjil(angka):
    if angka == 1:
        return 1
    else:
        return angka + hitung_ganjil(angka - 2)

#TESTCASE
print(hitung_ganjil(5))
print(hitung_ganjil(23))
print(hitung_ganjil(77))
```

Penjelasan :

1. Buat fungsi bernama **hitung_ganjil** dengan parameter bernama **angka**.
2. Jika angka berupa 1, maka program akan langsung mengeluarkan 1.
3. Jika bukan 1, maka disini rekursif terjadi. Fungsi akan menjumlahkan bilangan ganjil sekarang (angka) dengan hasil rekursif dari bilangan ganjil sebelumnya (angka - 2).

Output :

```
9
144
1521
```

SOAL 4

Source Code :

```
def total(angka):
    if angka < 10:
        return angka
    else:
        return (angka % 10) + total(angka // 10)
```

#TESTCASE

```
print(total(234))
print(total(12345))
print(total(2523))
```

Penjelasan :

1. Buat fungsi bernama **total** dengan parameter bernama **angka**.
2. Jika angka kurang dari 10, maka program akan langsung mengeluarkan output berupa angka tersebut.
3. Rekursif terjadi ketika **return (angka % 10) + total(angka // 10)** yang maksudnya mengambil digit terakhir dari angka dan menghapus digit terakhir dari angka / membulatkan hasil pembagiannya ke bawah.
4. Lalu ditambahkan untuk mendapatkan jumlahnya.

Output :

```
9
15
12
```

SOAL 5

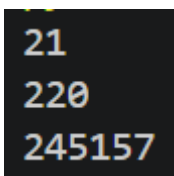
Source Code :

```
def kombinasi(n, k):  
    if k == 0 or k == n:  
        return 1  
    return kombinasi(n - 1, k - 1) + kombinasi(n - 1, k)  
  
#TESTCASE  
print(kombinasi(7, 5))  
print(kombinasi(12, 9))  
print(kombinasi(23, 7))
```

Penjelasan :

1. Buat fungsi bernama **kombinasi** dengan 2 parameter bernama **n** dan **k**.
2. Jika **k == 0 or k == n**, maka program akan selalu mengeluarkan output 1.
3. Rekursif disini menggunakan rumus dari segitiga pascal yaitu **$C(n,k) = C(n-1, k-1)$** ini berarti jumlah kombinasi yang termasuk elemen ke **n** lalu **ditambah** dengan **$C(n-1, k)$** yang berarti jumlah kombinasi yang tidak termasuk elemen ke **n**.

Output :



```
21  
220  
245157
```